02

The spec is the moat

The one page the AI keeps re-reading. Why drift, not bugs, is what kills your build. Template included.

SUBJECT	SPECS · DRIFT CONTROL	READER	SOLO BUILDERS ON AI TOOLS
AUTHOR	MICAH JONES	STATUS	CHAPTER TWO OF TEN
TIME	A TEN-MINUTE READ	REV	2026.08

§ 02.1

The killer isn’t bugs

Bugs are loud. They throw errors, fail the build, break the demo in front of your friend. You find bugs because bugs want to be found.

What kills builds is quieter. Somewhere around the fortieth session, you open your app the way a stranger would, and you feel it: everything works, and this is not the product you meant to build. Settings has eleven screens. There’s a dashboard you never asked for, grown from a “while I’m here” suggestion you approved on a tired Tuesday. The one flow that had to be effortless takes four taps.

Nothing is broken. That is exactly the problem.

DRIFT

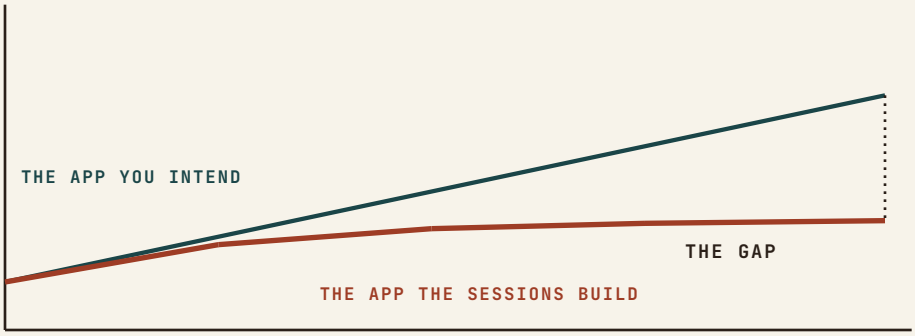
> The accumulating gap between the product you intend and the product the sessions are actually building. Not wrong code. Wrong direction, shipped correctly, one reasonable-looking diff at a time.

Chapter one gave you the invariant list, and that file is defense: it stops a session from undoing what already works. It does nothing to steer what gets built next. Steering is this chapter.

§ 02.2

Where drift comes from

Recall the mechanism. Every session starts by guessing what you are building. It reads your ask, samples whatever files it happens to open, and re-derives the product’s intent from the evidence. Forty sessions means forty independent guesses at what you want. Each one lands close. Close, compounded forty times, is far.



The intent lives in exactly one place the tool cannot read: your head. Every feature request you type is a translation from that head, made under deadline, at night, in fragments. The AI fills every gap in the translation with the most statistically ordinary choice. Your product drifts toward the average of every app it has ever seen, because that is what gap-filling means.

FIELD NOTE

Drift is why every half-finished AI build looks eerily alike: the same dashboard, the same card grid, the same settings sprawl. The average is what fills the gaps you left.

§ 02.3

One page, in the repo

The fix is the same move as chapter one, pointed at direction instead of correctness: put the intent where every session reads it before it reads anything else.

Not a product requirements document. Thirty pages is where intent goes to be skimmed. A spec that works in an AI-tool loop has one hard property: it fits in one page, so it gets read whole, every session, without eating the window. One page also forces you to decide what actually matters, which is most of the value before the tool ever sees it.

Six sections. Each is one to six lines.

WHAT. One sentence: the product, who it’s for, and the behavior that means it’s winning.

NOT. What this product refuses to be or do. The anti-scope.

SHAPE. The architecture in six lines or fewer: where data lives, who’s allowed to see what, what talks to what.

RULES. Your top invariants, or a pointer to the invariants file.

NOW. The current milestone, one line, with what “done” means for it.

LATER. The parking lot. Ideas go here to wait, instead of leaking into scope.

The strange one is NOT, and it does the most work. Drift rarely enters as a bad idea; it enters as a good idea that belongs to a different product. The AI is an enthusiastic yes-machine, and every “while we’re at it, should I add…” is a fork in the road. With a NOT section, that fork is not a judgment call at midnight. It is a lookup.

SPEC.md

```
# WHAT
Booking and payments for independent personal
trainers. One trainer, their clients, their
calendar. Winning = a client books and pays in
under a minute on a phone.
# NOT
Not a marketplace. Not multi-trainer gyms. No
social feed, no chat, no meal plans. No admin
dashboard beyond earnings and the calendar.
# SHAPE
Next.js on Vercel. Postgres w/ row-level security:
trainers see only their own clients. Stripe for
payments, webhooks own booking state. Twilio for
reminders. No other third parties.
# RULES
See CLAUDE.md invariants. Top three: bookings are
never double-writable; money state comes only from
Stripe webhooks; phone numbers are stored E.164 or
not at all.
# NOW
Milestone: first paying trainer. Done = she books
10 real sessions in a week without texting me for
help.
# LATER
Packages/subscriptions. Waitlists. Google Calendar
sync. Referrals.
```

Steal the shape, not the contents. Yours will be shorter in some sections and longer in NOT than you expect.

§ 02.4

Why this is the moat

Everyone has the same tools now. Same models, same editors, same starter templates, and the demos all look the same because of it. What separates the builds that become

products is not the tool. It is how little the builder lets the tool guess.

The spec is the one input the AI cannot generate, because it is the one thing only you know: what you actually want, and what you refuse to ship. Written down, it compounds. Every session starts aligned instead of guessing. Every proposal gets checked against NOT at the moment it's made, which is the cheap moment. Drift caught at proposal costs one sentence. Drift caught at regression costs a weekend, and chapter one showed you the weekend.

Same tools in everyone's hands. The page that says what you want is the only part they can't copy.

Two entries from my build log

Both of these are mine, both dated. The first cost six weeks. The second cost a week, and I wrote this manual partly because of it.

FROM THE BUILD LOG — ENTRY · 2026-05-19

Six weeks toward a reference nobody locked

On Ordani, I spent six weeks polishing the interface toward a quality bar that existed only as a feeling. Every session, the AI and I made real improvements against an imagined reference. The post-mortem line still stings: the work was optimized toward a direction that was never agreed, not even with myself.

The fix was not better design work. It was one page, written before the next round, that said what good means here: the reference products, the feel, and the anti-patterns to refuse. The page took an evening. The six weeks did not come back.

FIELD NOTE

The tools themselves are converging on this idea: plan modes, rules files, project instructions. A written plan the model critiques before code is the same move wearing a feature name. The habit transfers to every tool you'll ever use.

§ 02.5

FROM THE BUILD LOG

ENTRY · 2026-08

Four redesigns in one week

My own consulting site, the redesign that produced the site you found this manual on. Four full design rounds in a single week, every one rejected, some within minutes. The instruction I had given: “make it better.” The AI obliged, four different ways, toward four different averages.

Round five started with a locked sentence: nicer than what exists, no cheap gimmicks, photos of real work, keep what already worked. A one-line WHAT and a three-item NOT. It shipped in two passes, and I love it. Same tool. Same week. The page was the difference.

§ 02.6

The ritual

The spec only works as a habit loop. Five moves:

PRE-FLIGHT · THE SPEC RITUAL — RUN TONIGHT

- [ ] **Write SPEC.md tonight.** Six sections, one page, hard cap. If it spills past a page, NOT and LATER are where the excess goes.
- [ ] **Open every session with it.** "Read SPEC.md and the invariants file, then the task." Ten seconds. Direction and defense, loaded before any code.
- [ ] **Frame asks against NOW.** "Per the NOW milestone, build X" beats "build X." It tells the tool what to optimize for and, just as useful, what not to gold-plate.
- [ ] **Treat every "should I also…" as a NOT lookup.** If it's in NOT, the answer is no and costs nothing. If it's genuinely new, it goes to LATER, deliberately, not into the diff.
- [ ] **Change the spec only in its own commit.** The spec is allowed to evolve; it is not allowed to drift. A spec edit that rides along inside a feature diff is drift with paperwork.

One page, six sections, read at every session start, guarded by a five-move ritual. That is the moat. The next chapter draws the one diagram the SHAPE section summarizes: the architecture you didn't draw, and where AI tools quietly cut corners in it.

NEXT · CHAPTER 03

The architecture you didn’t draw

The single diagram every solo build needs. Auth, data, storage, edge, third parties, and where AI tools quietly cut corners.